Predicting Airline Passenger Satisfaction

Companion Theory Document

Kaggle Playground Series S6E10 — October 2026

*This document covers what is **new** in this competition. It is built from the experiments that were actually run: every number in a "What Happened" box comes from `experiments.csv` or a screening log.*

For AUC-ROC, gradient boosting, cross-validation basics and Optuna, see the earlier theory documents.

Contents

1 What Transfers and What Does Not **1**

2 Domain Knowledge: Airline Passenger Satisfaction **1**
- 2.1 The Task . . . 1
- 2.2 Features and Their Meaning . . . 2
- 2.3 What Drives Satisfaction . . . 2
- 2.4 Delays . . . 2

3 Synthetic Data and the Original Dataset **3**
- 3.1 What "Synthetic" Means Here . . . 3
- 3.2 How Close Are the Two Datasets? . . . 3
- 3.3 Approach A: Append the Original Rows . . . 3
- 3.4 Approach B: Lookup Features . . . 3
- 3.5 What Did Not Extend the Idea . . . 4

4 Per-Value Encoding: The Main Lever **4**
- 4.1 Why Trees Need Help With Per-Value Effects . . . 4
- 4.2 Target Encoding and Its Traps . . . 5
- 4.3 CatBoost's Ordered Target Statistics . . . 5
- 4.4 Embeddings in the Neural Net . . . 6

5 The Models **7**
- 5.1 LightGBM . . . 7
- 5.2 XGBoost . . . 7

5.3 CatBoost . . . 7
5.4 The Neural Net . . . 7
5.5 Models That Added Nothing . . . 8

6 Validation Discipline **8**
6.1 The Setup . . . 8
6.2 The Decision Rule . . . 8
6.3 Screening and the Winner's Curse . . . 8
6.4 Cross-Validation Versus the Leaderboard . . . 9

7 Ensembling **9**
7.1 Diversity . . . 9
7.2 Hill Climbing (Ensemble Selection) . . . 9
7.3 Is the Blend Overfitting Its Weights? . . . 9
7.4 Refit on All the Training Data . . . 10

8 A Catalogue of Leakage Traps **10**

9 Pseudo-Labelling **11**

10 Public Prediction Pools **12**
10.1 What They Are . . . 12
10.2 The Stack . . . 12
10.3 Checks Before Stacking . . . 12

11 What Did Not Work, and Why **13**

12 The Full Journey **13**

1 What Transfers and What Does Not

Transfer from Earlier Documents

Fully transfers:

- Binary classification, $\hat{p} = P(\text{satisfied} = 1 \mid \mathbf{x})$, scored by AUC-ROC.
- Stratified K-fold cross-validation with out-of-fold (OOF) predictions.
- Gradient-boosted trees (LightGBM, XGBoost, CatBoost), logistic regression, Optuna with TPE.

New in this competition:

- A dataset made mostly of 0–5 *survey ratings*, plus delays with long tails.
- Synthetic data generated from a public *original* dataset, and the ways to use that original.
- *Per-value encoding* as the main lever: telling models what is typical for each exact value.
- Neural networks with embeddings, trained on an Apple GPU.
- Ensembling discipline: blend gain versus single-model gain, nested CV, refitting on all rows.
- A catalogue of leakage traps, and public prediction pools.

The Lesson of This Competition

New **information** beat more **tuning**. The three largest gains all came from giving models per-value information they could not build themselves. Tuning, seeds and model variations added about +0.0001 each at best.

2 Domain Knowledge: Airline Passenger Satisfaction

2.1 The Task

Each row is a passenger. The label `satisfaction` is `True` (satisfied) or `False` (neutral or dissatisfied). The training set has 699,635 rows with a positive rate of 0.4436; the test set has 299,844 rows. The classes are nearly balanced, so class weighting is not a concern here.

2.2 Features and Their Meaning

Feature	Meaning	Type
Gender	Male / Female	2-level categorical
Customer Type	Loyal / disloyal customer	2-level categorical
Type of Travel	Business travel / Personal travel	2-level categorical
Class	Business / Eco / Eco Plus	3-level categorical
Age	Passenger age (75 distinct values)	Integer
Flight Distance	Distance flown (≈3,500 distinct values)	Integer
13 service ratings	Wifi, time convenience, online booking, gate location, food and drink, online boarding, seat comfort, entertainment, on-board service, leg room, baggage handling, check-in, cleanliness	Integer 0–5
Departure Delay	Minutes late on departure (mostly 0)	Integer, long tail
Arrival Delay	Minutes late on arrival; 292 missing in train, 130 in test	Integer, long tail

2.3 What Drives Satisfaction

In LightGBM's gain importance, **Online boarding** alone carried roughly a third to a half of all the gain, followed by **Inflight wifi service** (≈12%), then **Type of Travel** and **Class**. Digital touchpoints dominate; business travellers in business class are the easiest "satisfied" group.

The Rating Scale Is Not Purely Numeric

In the original dataset's description, a rating of 0 means *not applicable* (for example, no wifi on the flight), not "worse than 1". A rating is therefore better thought of as a small category than as a number. This is one reason why treating values as categories worked so well later on.

2.4 Delays

Both delays are 0 for most passengers and have very long right tails. Arrival delay is close to departure delay, so the two carry largely the same information. Because most values are 0, any method that bins by quantiles collapses: we asked for 32 quantile bins of the delays and got only 5 distinct groups (Section 4.4).

Comprehension Checkpoint

1. Why is a rating of 0 better treated as its own category than as "a bit worse than 1"?
2. The classes are 44% / 56%. Why does this make class weighting irrelevant for AUC here?
3. Why do quantile bins fail for a column where 80% or more of the values are 0?

3 Synthetic Data and the Original Dataset

3.1 What "Synthetic" Means Here

The competition data was generated by a model trained on the public *Airline Passenger Satisfaction* dataset (129,880 rows once its train and test files are combined, positive rate 0.4345). The generator copies the original's distributions approximately, not exactly. The original also has a 14th rating, *Inflight service*, that the competition dropped.

3.2 How Close Are the Two Datasets?

Three diagnostics shaped everything that followed:

- Exact row copies: only 0.002% of competition rows match an original row on all 21 features.
- Full rating-and-category profile (17 columns): 27% of competition rows match an original row.
- Single values: every individual value (every age, distance, delay) appears in the original.

3.3 Approach A: Append the Original Rows

The obvious idea is to add the 129,880 original rows to every training fold (never to validation).

What Happened in This Competition

- LightGBM with the original rows appended (exp02): blend −0.00022. **Worse.**
- Neural net with the rows and an `is_original` flag: −0.00006. **No help.**
- CatBoost (all columns categorical) with the rows and the flag: +0.00019 alone, +0.00010 in the blend, both screened folds better. It became a blend member (weight 0.20 in exp13).

The original data helps the model whose per-value statistics benefit from more rows (CatBoost), but not the others.

3.4 Approach B: Lookup Features

Instead of training on the original rows, *describe* each competition row by what happened to similar passengers in the original. For a value (or combination of values) c:

$$\text{lookup}(c) \;=\; \frac{\sum_{i \in O_c} y_i \;+\; m \cdot \bar{y}_O}{n_c + m}$$

where O_c is the set of original rows with value c, $n_c = |O_c|$, $\bar{y}_O$ is the original's overall satisfaction rate and m is a smoothing pseudo-count (we used $m = 20$).

Why Lookups Cannot Leak Across Folds

The labels in the formula come only from the *original* dataset. No competition label is used, so a lookup column computed once for all competition rows is safe to use in every fold.

What Happened in This Competition

LightGBM screen (learning rate 0.1, one seed, all 5 folds), against the raw features at 0.95970:

Lookup set	OOF AUC	Change	Folds better
A: one lookup per column (21)	0.95996	+0.00026	4/5
B: 17-column profile (+ match count) and category combination	0.95983	+0.00013	4/5
C: pairs of the top features	0.95971	+0.00001	3/5
A + B	**0.96004**	**+0.00034**	**5/5**

Full run (exp09): blend CV +0.00038, public leaderboard +0.00057 — the biggest public gain of the competition.

3.5 What Did Not Extend the Idea

Variations That Failed

- **Coarser profiles** (ratings only, in-flight ratings, ground ratings), which match 41%, 90% and 99% of rows: −0.00033, worse on all 5 folds. Likely cause: close copies of the *original's* rating patterns, which the synthetic data reproduces only approximately.
- **A model trained on the original** (CV AUC 0.995 on the original itself), used as a feature: LightGBM +0.00013, blend +0.00005. Below the bar.
- **Smoothing sweep** $m \in \{1, 5, 20, 50, 100, 200\}$: flat within ±0.00004 from 1 to 50; $m = 200$ lost 0.00016. Common values dominate, and for them m barely matters.
- **Predicting the dropped "Inflight service" rating** from the other columns: it is fairly predictable ($R^2 = 0.62$), so it adds little new information. LightGBM −0.00008.

Comprehension Checkpoint

1. Write the smoothed lookup formula. What does it return when $n_c = 0$? When n_c is very large?
2. Why can lookup features built from the original's labels never leak competition labels across folds?
3. Appending original rows hurt LightGBM but helped CatBoost. Suggest a reason based on how each model uses the data.
4. Why might a profile that matches 99% of rows be *less* useful than one that matches 27%?

4 Per-Value Encoding: The Main Lever

4.1 Why Trees Need Help With Per-Value Effects

A tree splits a numeric column at thresholds: $x \leq t$. That is efficient when the effect of x changes smoothly, but expensive when individual values behave differently (a particular flight

distance, a particular delay). Representing "these 40 scattered values are good, those 30 are bad" takes many splits, and the tree may never find them.

Two Corrections to the Churn Document

1. **Trees do use the order of label-encoded categories.** A split $x \leq t$ can only separate categories in their numeric order. Grouping non-adjacent categories needs extra splits. Native categorical handling (CatBoost, LightGBM's categorical features) avoids this.
2. **Statistics that use no labels cannot leak labels.** Medians, counts and frequencies computed on train and test together are not target leakage. Leakage requires *labels* to reach a feature or a decision that is then scored on those same rows. This competition used label-free frequencies over train and test safely.

4.2 Target Encoding and Its Traps

Target encoding replaces a value with the average label for that value. Computed on all training rows, it leaks each row's own label into its feature. The standard fix is K-fold encoding: for each fold, encode using the other folds only.

Target Encoding on the Competition Data

Out-of-fold encodings of the 17-column profile (with inner folds for the training rows):

- With match counts: -0.00168, worse on all 5 folds.
- Rates only: -0.00031.
- Single columns, long-tailed four only: LightGBM $+0.00011$, but blend $+0.00001$.

The counts were the trap: training rows were encoded from 80% of the fold's rows, validation rows from 100%, so every count was about 25% larger on validation rows than on the rows the model learned from.

Encoding Scale Must Match

Any feature computed differently for training rows and scored rows (different sample sizes, different data) has a different *distribution* in the two. Counts are the clearest case. Check that each engineered column has the same distribution where it is learned and where it is applied.

4.3 CatBoost's Ordered Target Statistics

CatBoost encodes categories internally. For each row i in a random permutation, it uses only rows that come *before* i:

$$\text{encode}_i(c) = \frac{\sum_{j<i,\, x_j=c} y_j + a \cdot p}{\sum_{j<i,\, x_j=c} 1 + a}$$

where p is the prior and a a smoothing weight. A row's own label never enters its own encoding, and every row is encoded the same way, so there is no scale mismatch. CatBoost also builds and encodes *combinations* of categorical columns automatically.

CatBoost With Every Column Categorical (exp10)

Treating all 21 raw columns as categories — including Age, Flight Distance and both delays — made CatBoost the best single model (0.96072 against LightGBM's 0.96046) and lifted the blend by +0.00041 (public +0.00049).

A speed test then showed *where* the gain comes from (fold 1, learning rate 0.15, reference 0.96073):

- Long-tailed columns kept numeric: −0.00099 — the biggest effect measured.
- Combinations capped at 2 or 3 columns: −0.00021 / −0.00015.
- Half the features per split: −0.00002, and no faster.

4.4 Embeddings in the Neural Net

A neural net handles per-value effects with an *embedding*: a small learned vector for each value of a column.

Bins Versus Exact Values

The first neural net used 32 quantile bins for each delay. Because most delays are 0, the bin edges collapsed and the delays got only **5** distinct groups. Giving one embedding per exact value (491 and 493 groups for the delays, about 5,000 for distance) improved the neural net from 0.95938 to **0.96026** (+0.00088, 5/5 folds) and the blend by +0.00013 (public +0.00027).

Blend Gain Is What Counts

Long-tailed columns as categories in LightGBM made LightGBM *worse* on its own (−0.00022) but improved the blend (+0.00013) by making it less like CatBoost. Single-column target encodings made LightGBM *better* (+0.00011) but added nothing to the blend (+0.00001), because CatBoost and the neural net already carried that information. Always judge a change by what it adds to the blend.

Comprehension Checkpoint

1. Explain in one sentence why a tree needs many splits to capture "value-specific" effects.
2. Why did match counts make out-of-fold target encoding so much worse than no encoding?
3. How does CatBoost's ordered encoding avoid leaking a row's own label?
4. Why did 32 quantile bins produce only 5 groups for the delays?
5. Give an example from this competition where a change helped one model but not the blend, and explain why.

5 The Models

5.1 LightGBM

Tuned with Optuna (TPE) at a high learning rate (0.1) on 3 folds, then trained at 0.02 with 5 seeds averaged.

`num_leaves`	80	`min_child_samples`	7
`colsample_bytree`	0.57	`subsample`	0.96
`reg_lambda`	0.17	`reg_alpha`	0.46
`min_split_gain`	0.33	`max_bin`	1023
`path_smooth`	12.1	`cat_smooth`, `cat_l2`	16.4, 2.8

What Happened in This Competition

Tuning (exp03), then XGBoost and seeds (exp04), then a wider search with a lower learning rate and 5 seeds (exp05) took the blend from 0.95899 to 0.96014 and the public score from 0.95888 to 0.95942.

5.2 XGBoost

Tuned the same way. It scored well alone (0.96001 with the lookups) but received almost no blend weight: it is too similar to LightGBM to add information.

5.3 CatBoost

All raw columns categorical, depth 7, $\ell_2 = 3$. Seven tuning variants (depth 6/8, ℓ_2 1/10, combinations up to 5 columns, Bernoulli subsampling, more split randomness) all scored at or slightly below the defaults. On CPU, the learning rate was the only real speed lever: at 0.15 a fold took 2.6 minutes against about 11 at 0.06, for about -0.0001. Two fast seeds (0.96077) beat one slow seed (0.96072).

5.4 The Neural Net

- **Inputs:** an embedding for each category, each rating and each exact Age, distance and delay (size 8 for columns with up to 10 values, 4 for larger ones); the numeric columns as well, log-transformed where skewed, median-filled and standardised; a missing-arrival-delay flag.
- **Network:** MLP $256 \to 128$ with BatchNorm, SiLU and dropout 0.27.
- **Training:** AdamW (weight decay 3.6×10^{-4}), one-cycle learning rate peaking at 2.1×10^{-3}, batch 1024, 25 epochs; the epoch with the best validation AUC is kept, like early stopping in trees.
- **Ensembling:** 5 seeds averaged. Trained on the Apple GPU (MPS).

A macOS Pitfall

PyTorch and LightGBM/XGBoost each load their own OpenMP runtime on macOS. Used in one process, they can deadlock the first time PyTorch creates a tensor. Calling `torch.set_num_threads(1)` on macOS avoids it; training on the GPU is unaffected.

5.5 Models That Added Nothing

- LightGBM variants (ratings as categories, `extra_trees`, `linear_tree`): 0.999 correlated with the main LightGBM, blend +0.00001 at most.
- Logistic regression on one-hot values (with and without pairs): 0.951 / 0.955 alone, zero blend weight.
- Neural-net architecture variants (wider embeddings, cross-feature layer, bigger layers, slower schedule): all equal or worse.

Comprehension Checkpoint

1. Why tune at learning rate 0.1 and train the final model at 0.02?
2. XGBoost scored 0.96001 alone yet got zero blend weight. How can a strong model add nothing?
3. Why is keeping the best validation epoch the neural-net equivalent of early stopping?
4. What does averaging 5 seeds reduce, and why does it help a blend?

6 Validation Discipline

6.1 The Setup

- **Fixed folds**, saved once (`folds_seed42.csv`), so every experiment is compared on identical rows.
- **An experiment log** with one row per model: fold scores, OOF score, change against a baseline, folds improved, runtime, device, notes and the public score.
- **Saved predictions** (OOF and test) for every model, so unchanged models are reused, not retrained.

6.2 The Decision Rule

A change is adopted only if it gains at least +0.0002 OOF AUC *and* improves at least 4 of the 5 folds. Fold-level consistency matters as much as the average: a gain on every fold is unlikely to be noise.

6.3 Screening and the Winner's Curse

Cheap screens (learning rate 0.1, one seed, 2 or 5 folds) rank ideas quickly. But choosing the best of many candidates on the same few folds is biased upwards: the winner is partly the luckiest.

The Neural-Net Search That Shrank

An Optuna search on 2 folds with 1 seed found settings scoring 0.95970 against 0.95883: +0.00087. Rerun with 5 seeds on all 5 folds, the gain was +0.00003. Seed noise and selection among 20 trials explained almost all of it.

Warning

Confirm every screened winner at full settings — all folds, several seeds — before believing its size.

6.4 Cross-Validation Versus the Leaderboard

The public leaderboard scores only part of the test set (about 20%), so differences of about 0.0001 there are noise. CV uses all 700,000 training rows and is the better guide — as long as it tracks the leaderboard.

Experiment	CV	Public	CV − public
exp03 tuned LightGBM	0.95957	0.95888	0.00069
exp05 wider tuning, seeds	0.96014	0.95942	0.00072
exp08 refit	0.96023	0.95962	0.00061
exp09 original lookups	0.96061	0.96019	0.00042
exp10 all-categorical CatBoost	0.96102	0.96068	0.00034
exp11 exact-value embeddings	0.96115	0.96095	0.00020
exp13 tuned NN, CatBoost with original rows	0.96129	0.96100	0.00029

Key Insight

The gap stayed stable or shrank, and every CV gain of 0.0003 or more showed up on the leaderboard. That is the evidence that CV can be trusted for decisions, and that the private leaderboard will likely follow it.

Comprehension Checkpoint

1. Why save folds to a file instead of re-splitting with the same seed each time?
2. Explain the winner's curse using the neural-net search as the example.
3. Why is the public leaderboard noisier than 5-fold CV on 700,000 rows?
4. What would it mean if the CV − public gap suddenly grew after a change?

7 Ensembling

7.1 Diversity

Every model here is more than 0.99 correlated with every other. A blend improves only where models make *different* mistakes, which is why model type and encoding mattered more than model strength.

7.2 Hill Climbing (Ensemble Selection)

Start from the best single model. At each step, add one more "vote" for whichever model most improves the blend's OOF AUC (models can be picked repeatedly). The final weights are the vote shares at the best step. Final blend weights: neural net 0.31, LightGBM 0.22, fast CatBoost 0.20, CatBoost with original rows 0.20, CatBoost 0.07.

7.3 Is the Blend Overfitting Its Weights?

The weights are chosen on the same OOF predictions they are scored on. A fair check fits the weights on 4 folds and scores the 5th:

What Happened in This Competition

Hill climbing scored 0.96114 this way, against 0.96115 when fitted on all folds — no meaningful optimism. A logistic regression stacker on the models' log-odds scored the same (0.96114); a small LightGBM stacker was worse (0.96107).

7.4 Refit on All the Training Data

CV models each train on 80% of the rows. For the final submission, retrain each model on 100%:

- Trees: fixed tree count = mean best iteration across folds $\times$ 1.15 (25% more rows supports more trees).
- Neural net: keep the full learning-rate schedule but stop at the median best epoch.
- Blend the refit predictions with the CV blend weights.

What Happened in This Competition

Refit against fold-averaged, on the public leaderboard: +0.00008, +0.00008, +0.00017, +0.00013, +0.00016 — five for five. CV cannot score a refit, so the leaderboard is the only check.

Comprehension Checkpoint

1. Why can two models scoring 0.9605 blend to 0.9612, while adding a third at 0.9600 adds nothing?
2. Describe hill climbing in your own words. Why allow a model to be picked more than once?
3. How do you check that blend weights are not overfitted to the OOF predictions?
4. Why multiply the best iteration by 1.15 when refitting on all rows?

8 A Catalogue of Leakage Traps

Leakage makes CV look better (or sometimes worse) than the leaderboard will. Every trap below came up in this competition.

Trap	What goes wrong	Fix used
Target encoding on all rows	A row's own label enters its feature	K-fold encoding; CatBoost's ordered encoding
Count-scale mismatch	Encoded counts differ between training and validation rows (-0.00168)	Drop counts, or encode both the same way
Lookup features for original rows	Original rows' lookups contain their own labels	CatBoost with original rows used raw columns only
Blend weights fitted on OOF	Weights tuned on the rows they are scored on	Nested check: fit on 4 folds, score the 5th
Pseudo-label teacher	A teacher that saw fold k passes its labels to the fold-k student	A separate teacher per fold, trained without that fold
Stack of stacks	A published stack was fitted on all rows, so as a member it inflates nested CV	Never use a published stack as a member
Winner's curse	The best of many screened candidates is partly luck	Confirm at full folds and seeds

Key Insight

Two questions catch most leakage: *did any label reach a feature or a decision that is scored on the same row?* and *is every feature computed the same way where it is learned and where it is applied?*

Comprehension Checkpoint

1. Pick two traps from the table and explain the mechanism in your own words.
2. Why does the count-scale mismatch *lower* the CV score, while most leakage raises it?
3. Why must a pseudo-labelling teacher be trained without the fold it is used for?

9 Pseudo-Labelling

Pseudo-labelling adds test rows to training, labelled with a model's predictions. Done fold by fold to avoid leakage: for fold k, a teacher (LightGBM + CatBoost) trains on the other folds, its test predictions become the pseudo-labels, and a LightGBM student trains on the other folds plus the labelled test rows.

What Happened in This Competition

- **Hard labels** (only test rows with $p \geq 0.98$ or ≤ 0.02, about 5%): -0.00003.
- **Soft labels** (all test rows, labelled with the teacher's probability, cross-entropy loss): LightGBM $+0.00028$ on all 5 folds — but blend $+0.00002$.

The student mostly learned what the teacher knew, and half the teacher was CatBoost, which was already in the blend.

Comprehension Checkpoint

1. Why did soft labels help LightGBM alone but not the blend?
2. Why do confident pseudo-labels (hard labels) teach the model almost nothing?

10 Public Prediction Pools

10.1 What They Are

In recent Playground competitions, competitors publish their OOF and test predictions on shared folds. Others stack hundreds of them. For S6E10 the shared folds were a shuffled, stratified 5-fold split with `random_state=42` — identical to ours, so our predictions plugged straight in.

10.2 The Stack

A logistic regression on the members' log-odds z_{ij} with strong ℓ_2 regularisation:

$$\min_{\mathbf{w},b} \sum_i \text{logloss}\Big(y_i,\ \sigma\big(b + \textstyle\sum_j w_j z_{ij}\big)\Big) \ + \ \frac{1}{2C}\sum_j w_j^2, \qquad C = 0.003.$$

Family-wise regularisation multiplies some columns by a factor s before fitting. Since $w_j z_{ij} = (w_j/s)(s\, z_{ij})$, the effective penalty on that column becomes w_j^2/s^2: scaling by 3 divides the penalty by 9. The tabular foundation-model columns (TabPFN) were scaled by 3, others by 1.5. The final stack was 0.8 of this logistic model plus 0.2 of a small LightGBM stacker, each standardised.

10.3 Checks Before Stacking

Row alignment, probability versus log-odds scale, train/test drift, OOF AUC above 0.9625 ("too good to be out of fold"), and near-copies (rank correlation above 0.9999 with an earlier member: 45 dropped).

What Happened in This Competition

- 293 public pairs loaded, 247 kept. The reproduction matched the published notebook to six decimals (0.962211 plain logistic, 0.962228 blended).
- Adding our six models changed nothing (0.962227): the pool already covered all-categorical CatBoosts, RealMLPs and original-data members.
- Nested CV 0.96222, against 0.96129 for our own best blend.

The Trade-Off

Most of that gain is other people's work, and many competitors submit nearly the same public stack, so it gives little edge in the private ranking. It is allowed and common in Playground competitions; whether it is the result you want is a personal choice. A sensible pair of final picks is the pool stack plus your own best blend, which are genuinely different.

Comprehension Checkpoint

1. Why must every member of a stack share the same folds?
2. Show why multiplying a column by s divides its effective ℓ_2 penalty by s^2.
3. Why is a published stack a dangerous member of your own stack?

11 What Did Not Work, and Why

Idea	Result	Likely reason
Hand-made features (rating summaries, delay gap, bins, Class × Travel)	blend −0.00018	Trees build these themselves; extra columns add noise
Original rows appended to LightGBM	blend −0.00022	Distribution differs; LightGBM cannot separate the sources
Coarser original profiles	−0.00033	Copies the original's patterns, not the synthetic data's
Target encoding on competition data	−0.00031 to −0.00168	Noise and count-scale mismatch
LightGBM variants	$\leq +0.00001$	0.999 correlated with the main LightGBM
CatBoost tuning	all ≤ 0	Defaults already near the best
Neural-net architecture variants	all $\leq +0.00003$	Current design already well set
Logistic regression on one-hot values	zero weight	Too weak to contribute
Stacking our own models	±0	Hill climbing already optimal here
Pseudo-labelling	blend +0.00002	Distils information already in the blend

12 The Full Journey

Step	Change	CV	Public
Start	First submission	—	0.95853
exp00	LightGBM + CatBoost baseline, fixed folds	0.95899	—
exp03	Tuned LightGBM	0.95957	0.95888
exp04	Tuned XGBoost; 3 seeds	0.95974	0.95896
exp05	Wider tuning, learning rate 0.02, 5 seeds	0.96014	0.95942
exp06	Neural net in the blend	0.96020	0.95946
exp07	Binned embeddings	0.96023	0.95954
exp08	Refit on all rows	0.96023	0.95962
exp09	**Original-dataset lookups**	0.96061	0.96019
exp10	**All-categorical CatBoost**	0.96102	0.96068
exp11	Exact-value embeddings	0.96115	0.96095
exp12	Extra CatBoost and neural-net seeds	0.96125	0.96097
exp13	Tuned neural net; CatBoost with original rows	0.96129	0.96100
exp14	Stack of 247 public predictions	0.96222	—

Lessons Worth Keeping

1. **New information beats tuning.** Look for what the models cannot see, especially per-value effects.
2. **Judge by blend gain**, not single-model gain.
3. **Trust CV that tracks the leaderboard**, and ignore leaderboard moves under about 0.0001.
4. **Screen cheaply, confirm fully.** Small screens find ideas; full runs size them.
5. **Ask the two leakage questions** about every feature, blend and teacher.

Theory First. Math Second. Code Last.

Same principles, new domain — and this time, every claim was tested.